

Before You Start

1 Binaries	Download the pre-built spiritbun TurboQuant package or use the bundled Update Binaries button. It contains llama-server.exe, llama-bench.exe, llama-cli.exe and all ggml-*.dll / llama.dll files.
2 NVIDIA GPU	Install CUDA Toolkit 12.x or 13.x (nvidia.com/cuda). Copy the matching cuBLAS DLLs (cublas64_12.dll + cublasLt64_12.dll for CUDA 12, or cublas64_13.dll + cublasLt64_13.dll for CUDA 13) into the llama-server folder.
3 AMD GPU	Install ROCm 6.x (rocm.docs.amd.com). Use the HIP/ROCm build variant of llama-server. No extra DLLs needed — ROCm runtime ships its own libraries.
4 macOS (Apple Silicon)	Metal is built-in — no extra drivers or DLLs required. Use the macOS build variant. Works on M1 / M2 / M3 / M4.
5 First model	Download a GGUF model from Hugging Face. Good starter: meta-llama-3.1-8b-instruct-q4_k_m.gguf (~4.6 GB). Bigger / better: Qwen3.5-9B-UD-Q4_K_XL.gguf (~5.6 GB). Place the file anywhere, then add the folder in QLauncher → Paths.

⚠ CUDA version compatibility

CUDA 12.4–12.8: fully tested, recommended. CUDA 12.8+ required for RTX 5090 (sm_120). CUDA 13.2: experimental, may work. CUDA 13.1: DO NOT USE — known MMQ kernel segfault that causes crashes during inference. If you need CUDA 13.x, use 13.2 or newer.

💡 Where to find the CUDA DLLs (Windows)

Start → search for "NVIDIA Control Panel" → Help → System Information shows your driver version. Then download the matching CUDA Toolkit. The two required DLLs are in the bin\ subfolder. Alternatively, copy them from any machine that already has CUDA 12.x installed.

What is TurboQuant QLauncher?

TurboQuant QLauncher is a lightweight desktop GUI for managing llama-server with TurboQuant KV-Cache compression. Launch large language models on any NVIDIA, AMD, Intel or Apple GPU with a single double-click, switch between models and binary builds instantly, run benchmark matrices, and monitor VRAM and system-RAM usage — all without touching the command line. New in v0.50: the Probe (qnut) tool finds the best Q / N / C settings for any (model, server) combination automatically.

💡 Why TurboQuant?

Standard llama.cpp stores the KV-cache in full float16 precision. TurboQuant compresses it using Walsh–Hadamard rotation (ICLR 2026, Google Research), achieving 4–5× smaller KV-cache with minimal quality loss. This means longer context, more VRAM headroom — or fitting a bigger model on the same hardware.

Key Features

 One-click Launch	Double-click any model row to start llama-server on that GPU. Double-click again to stop.
 Probe (NEW v0.50)	Read-only differential probe: runs all KV × LA combinations against an f16 baseline and finds the three Q / N / C anchors per (model, server) combination. One-time per combination, then results are cached and instantly available via the Q / N / C mode buttons.
Q / N / C Modes (NEW v0.50)	Three mode buttons next to the KV row. After running Probe, click Q (Quality), N (Neutral) or C (Context) to apply the recommended (KV, LA) tuple in one click. The KV/LA buttons visibly move so you can see what each mode means.
 Live VRAM Bars	Braille-style bars show effective VRAM usage per GPU and CPU RAM — updated live when you switch KV config or LA mode.
 KV-aware Colors	Green = fits comfortably. Amber = tight. Red = overflow. Bars and ✓ / ✗ markers update with the compression selection.
 Benchmarking	Built-in llama-bench: single config or Bench All matrix (KV × LA in one run). f16 baseline + deltas, saved as Markdown.
 Status Pill	Header pill shows ● Idle, ⌚ Loading while the model loads into VRAM, then ● Running (click to open the browser).
 Persistent Config	Window position, splitter, KV selection, port, ctx, timeout, model paths, slot bookmarks, and Probe profiles are saved automatically on exit.
 Multi-Directory Scan	Scan up to 3 separate model directories together. Optional recursive subdirectory walk. Duplicates are detected automatically.
 Quick-Switch Slots	Bookmark up to 6 llama-server.exe builds (mainline / TheTom / spiritbuun / Gemma4 / etc.). One click in the footer switches the active build.
 Layer-Adaptive	Keep critical layers (first / last) at q8_0 for best quality. Modes 1, 5, 7 via LA buttons, passed as TURBO_LAYER_ADAPTIVE env var.
 Bench All Matrix	Matrix benchmark: every selected KV config × every selected LA mode in one run. Stoppable between runs. Per-run timeout configurable.
 Universal GPU Detection	NVIDIA (nvidia-smi), AMD (rocm-smi), Intel and Apple Silicon (WMI / lspci / sysctl). Mixed-architecture systems are supported.

Header Controls

KV row	Six KV-cache modes: f16, q8 ₀ +t4, t3/t3, t4/t4, q8 ₀ +t3, q8 ₀ /q8 ₀ . Click to select; live VRAM bars and fit indicators update immediately.
Q / N / C (NEW v0.50)	Three small mode buttons between the KV row and the Probe button. Greyed out until a Probe profile exists for the current (model, server) combination. Click to apply the corresponding (KV, LA) anchor. Manual KV/LA clicks reset the active mode.
 Probe (NEW v0.50)	Opens the Probe Model dialog. Runs the three-phase qnut workflow against the currently selected model on the current server slot. Read-only — the model is never modified, only observed. Duration: ~10–25 minutes one-time per (model, server). Results are cached as a profile.

Port	TCP port for llama-server (default 8080). Change here before launch.
Ctx	Context window in tokens. Empty = use the model's default. Any value is passed to llama-server as -c <ctx>.
Timeout	Per-run timeout for llama-bench in seconds (default 90). Prevents hangs in Bench All matrices.
☐ No Thinking	Disable reasoning / thinking mode via --reasoning off. Useful for tool-calling models.
☐ Benchmark	Single-config benchmark mode. Double-click or Run executes f16 baseline + the currently selected KV config and writes a delta-% table.
☐ Bench All	Activates the full KV × LA matrix. All KV and LA buttons turn blue; click any to deselect. Run starts, Stop cancels after the current run.
LA: off / 1 / 5 / 7	Layer-Adaptive modes: off (uniform), 1 (boundary-V), 5 (first4 + last4 at q8_0), 7 (aggressive). In Bench All they toggle individually.
Run / Stop	Run starts the benchmark with the current selection. Stop cancels after the in-flight run completes.
• Status pill	Idle / Loading / Running. Click while Running to open http://localhost:{port} in the browser.

Probe (qnut) — NEW in v0.50

Probe is a read-only differential measurement tool. It does not change anything about the model or the hardware — it just runs the model under each KV+LA combination, observes the responses, and writes a profile. After the profile is built, the three Q / N / C mode buttons in the top bar become active and let you apply the best settings with one click.

The underlying methodology — comparing compressed configurations against an f16 baseline rather than against hardcoded expected answers — is an established technique in the quantization research community and not original to QLauncher. What is original here is the specific implementation: the three-phase workflow, the (KV, LA) tuple anchors, the per-(model, server) profile storage, and the Q / N / C mode-button UI.

Why differential?

The naive approach is to ask the model a question, hardcode the expected answer, and check whether the model produced it. This breaks on small or weakly-quantised models that struggle even on plain f16 — every config looks broken because the model itself cannot answer reliably. The differential approach uses the f16 baseline as the reference instead: each compressed config is credited if it produces the same answer as f16, regardless of whether that answer was "correct". A dumb model is dumb consistently across configs, and the test correctly identifies the configs as KV-transparent.

The three phases

Phase 0 Baseline	f16 + LA=0 — collect responses for all probes. No judgement; we are just establishing the reference. ~90 seconds.
Phase 1 KV sweep	5 remaining KV configs at LA=0. Each response is compared to the baseline. Score 0–3 per config. ~7–8 minutes.
Phase 2 LA sweep	For every KV that scored ≥2 in phase 1, the three other LA modes (1, 5, 7) are tested too. Plus f16 with all LAs to verify Lookahead transparency on the baseline. ~9–18 minutes.

How the anchors are derived

Quality (Q)	Always (f16, LA=0) by definition. f16 is the reference baseline; nothing can be more faithful than the baseline itself.
Neutral (N)	The strongest-compression tuple with score 3 (perfect baseline match on all probes). This is the everyday driver — maximum compression that still behaves indistinguishably from f16.
Context (C)	The strongest-compression tuple with score ≥ 2 (one probe may diverge). Use this when you need every megabyte of KV cache for long context. Quality is slightly relaxed but the model still produces usable output.
degraded flag	If no tuple in the entire matrix achieves score 3, the run is marked degraded and Neutral falls back to the best available score. The dialog warns you when this happens — it usually means the model is small / heavily quantised and is sensitive to any KV compression.



Profile storage

Profiles are stored per (model, server_slot) under `cfg["nut_profiles"]`. The same model on different server builds (mainline / TheTom / spiritbuun / Gemma4) gets independent profiles — the build matters because the KV-cache implementation differs between forks. Switching server slot via the footer buttons re-evaluates the Q / N / C buttons automatically.



Recommended workflow

1) Add a model and select it. 2) Pick the GPU row you intend to use. 3) Click  Probe and wait — typically 10–25 minutes per model+server combination. 4) After the run, Q / N / C light up. 5) From now on, just click Q for quality work, N for everyday use, C for long-context work. The KV and LA buttons follow visibly so you learn what each mode means. 6) Re-run Probe only when you upgrade the server build or add a new model.

Footer Controls

Rescan Models	Rescan all configured model directories now. Honours the recursive flag.
Update Binaries	Run the bundled binary updater for the active llama-server folder.
Slot buttons	One-click switch between bookmarked llama-server.exe builds. The active slot is highlighted. Switching a slot also re-evaluates the Probe profile for the new (model, server) combination.
 Save Results...	Turns green when a benchmark finishes. Click to append the result table to TurboQuant_Benchmark.md next to the launcher.
Paths	Open the Configure Paths dialog: model directories, recursive flag, active llama-server.exe, and quick-switch slots.
About	Version, license, build info, and links to upstream projects.

Mouse & Keyboard

Double-click row	Launch server with the selected model on that GPU (or stop if already running).
Single click	Select a model / GPU row without launching.
↑ ↓ / ← →	Navigate models (up / down) and GPUs within a model (left / right).
Enter	Toggle the server for the currently selected row.
Escape	Stop the running server.
Click status pill	Open <code>http://localhost:{port}</code> in the browser (only when Running).
Drag splitter	Resize model list vs. log area.

KV-Cache Configurations

f16	Baseline. Full float16 KV-cache, no compression. Used as reference for benchmark deltas and Probe baseline.
q8o+t4	K at q8_0, V at turbo4. Best general-purpose compression. $\approx 3.5\times$ smaller KV-cache, near-zero quality loss.
t3/t3	Both K and V at turbo3. Most aggressive that still preserves quality on most models. $\approx 4.5\times$ smaller.
t4/t4	Both K and V at turbo4. Maximum compression. Some models show measurable quality drop — verify with a Probe run.
q8o+t3	K at q8_0, V at turbo3. Conservative middle ground.
q8o/q8o	Both K and V at q8_0. The classic non-TurboQuant 8-bit baseline for comparison.



Don't pick KV manually — let Probe decide

The whole point of v0.50's Probe + Q / N / C workflow is that you don't have to memorise which KV config is best for which model. Run Probe once per (model, server), then just click Q, N, or C. The KV and LA buttons will visibly move to the recommended position, and over time you'll develop a feel for which models prefer which settings.

Open Source Projects & Acknowledgements

★ spiritbuun / llama.cpp	TurboQuant CUDA implementation · this build · github.com/spiritbuun/llama-cpp-turboquant-cuda
★ TheTom / turboquant_plus	Alternative TurboQuant build with fused mmvq kernel · github.com/TheTom/turboquant_plus
★ ggml-org / llama.cpp	Upstream llama.cpp project · github.com/ggml-org/llama.cpp
💬 Community Discussion	Benchmark findings · seanrasch · AmesianX · primoco et al. · github.com/ggml-org/llama.cpp/discussions/20969
🔬 Probe methodology	Differential f16-baseline testing is an established technique in the quantization research community. The TurboQuant paper (Zandieh et al., ICLR 2026) and ongoing work in Discussion #20969 informed the v0.50 implementation.

TurboQuant paper

Zandieh et al., "TurboQuant: KV Cache Compression via Walsh–Hadamard Rotation", ICLR 2026, Google Research. This tool is an independent community project, not affiliated with Google.

Voraussetzungen

1 Binaries	Lade das vorkompilierte spiritbuun-TurboQuant-Paket herunter oder verwende den eingebauten Update-Binaries-Button. Es enthält llama-server.exe, llama-bench.exe, llama-cli.exe und alle ggml-*.dll- / llama.dll-Dateien.
2 NVIDIA-GPU	Installiere CUDA Toolkit 12.x oder 13.x (nvidia.com/cuda). Kopiere die passenden cuBLAS-DLLs (cublas64_12.dll + cublasLt64_12.dll für CUDA 12 bzw. cublas64_13.dll + cublasLt64_13.dll für CUDA 13) in den llama-server-Ordner.
3 AMD-GPU	Installiere ROCm 6.x (rocm.docs.amd.com). Verwende die HIP/ROCm-Build-Variante von llama-server. Keine zusätzlichen DLLs nötig — die ROCm-Runtime liefert eigene Bibliotheken mit.
4 macOS (Apple Silicon)	Metal ist eingebaut — keine zusätzlichen Treiber oder DLLs erforderlich. Verwende die macOS-Build-Variante. Läuft auf M1 / M2 / M3 / M4.
5 Erstes Modell	Lade ein GGUF-Modell von Hugging Face. Guter Einstieg: meta-llama-3.1-8b-instruct-q4_k_m.gguf (~4,6 GB). Größer / besser: Qwen3.5-9B-UD-Q4_K_XL.gguf (~5,6 GB). Datei beliebig ablegen, Ordner dann in QLauncher → Paths eintragen.

⚠ CUDA-Versionskompatibilität

CUDA 12.4–12.8: vollständig getestet, empfohlen. CUDA 12.8+ erforderlich für RTX 5090 (sm_120). CUDA 13.2: experimentell, kann funktionieren. CUDA 13.1: NICHT VERWENDEN — bekannter MMQ-Kernel-Segfault, der Abstürze während der Inferenz verursacht. Falls CUDA 13.x benötigt wird, mindestens 13.2 oder neuer einsetzen.

💡 Wo finde ich die CUDA-DLLs? (Windows)

Start → „NVIDIA Control Panel“ suchen → Hilfe → Systeminformationen zeigt die Treiberversion. Passenden CUDA Toolkit herunterladen. Die zwei benötigten DLLs liegen im Unterordner bin\.

Alternativ: von einem Rechner kopieren, auf dem CUDA 12.x bereits installiert ist.

Was ist TurboQuant QLauncher?

TurboQuant QLauncher ist eine schlanke Desktop-GUI zum Verwalten von llama-server mit TurboQuant-KV-Cache-Komprimierung. Per Doppelklick startest du Sprachmodelle auf einer beliebigen NVIDIA-, AMD-, Intel- oder Apple-GPU, wechselst blitzschnell zwischen Modellen und Binary-Builds, führst Benchmark-Matrizen durch und überwachst VRAM- und System-RAM-Verbrauch — alles ohne Kommandozeile. Neu in v0.50: das Probe (qnut)-Tool findet die besten Q / N / C-Einstellungen für jede (Modell, Server)-Kombination automatisch.

💡 Warum TurboQuant?

Standard-llama.cpp speichert den KV-Cache in voller float16-Genauigkeit. TurboQuant komprimiert ihn mittels

Walsh–Hadamard-Rotation (ICLR 2026, Google Research) auf 4–5× kleinere Größe — bei minimalem Qualitätsverlust. Das Ergebnis: längere Kontextfenster, mehr VRAM-Spielraum oder Platz für größere Modelle auf derselben Hardware.

Hauptfunktionen

 Ein-Klick-Start	Doppelklick auf eine Modellzeile startet llama-server auf dieser GPU. Nochmal doppelklicken stoppt.
 Probe (NEU v0.50)	Read-only Differential-Probe: lässt alle KV × LA-Kombinationen gegen eine f16-Baseline laufen und findet die drei Q / N / C-Anker pro (Modell, Server)-Kombination. Einmalig pro Kombination, danach sind die Ergebnisse gecached und sofort über die Q / N / C-Mode-Buttons verfügbar.
Q / N / C-Modi (NEU v0.50)	Drei Mode-Buttons neben der KV-Reihe. Nach einem Probe-Lauf klick auf Q (Quality), N (Neutral) oder C (Context) und das empfohlene (KV, LA)-Tupel wird in einem Klick angewendet. Die KV/LA-Buttons bewegen sich sichtbar mit, damit man sieht was der jeweilige Modus bedeutet.
 Live-VRAM-Balken	Braille-Balken zeigen den effektiven VRAM-Verbrauch pro GPU sowie CPU-RAM — werden live aktualisiert beim KV- oder LA-Wechsel.
 KV-bewusste Farben	Grün = passt bequem. Gelb = knapp. Rot = Überlauf. Balken und ✓ / ✗ ändern sich mit der Kompressionsauswahl.
 Benchmarking	Integriertes llama-bench: Einzelconfig oder Bench-All-Matrix (KV × LA in einem Lauf). f16-Baseline + Deltas, gespeichert als Markdown.
 Status-Pille	Status-Pille im Header zeigt ● Idle, ⏳ Laden während das Modell in den VRAM geladen wird, dann ● Laufend (Klick öffnet Browser).
 Persistente Config	Fensterposition, Splitter, KV-Auswahl, Port, Ctx, Timeout, Modell-Pfade, Slot-Lesezeichen und Probe-Profil werden beim Schließen automatisch gespeichert.
 Mehrere Modell-Verzeichnisse	Bis zu 3 separate Modell-Verzeichnisse gleichzeitig scannen. Optionales rekursives Durchsuchen der Unterordner. Dubletten werden automatisch erkannt.
 Quick-Switch-Slots	Bis zu 6 llama-server.exe-Builds als Lesezeichen (mainline / TheTom / spiritbuun / Gemma4 / etc.). Ein Klick im Footer wechselt den aktiven Build.
 Layer-Adaptive	Kritische Layer (erste / letzte) bei q8_0 halten für beste Qualität. Modi 1, 5, 7 über LA-Buttons, wird als TURBO_LAYER_ADAPTIVE-ENV-Variable gesetzt.
 Bench-All-Matrix	Matrix-Benchmark: jede gewählte KV-Config × jeder gewählte LA-Modus in einem Lauf. Zwischen Runs stoppbar. Per-Run-Timeout konfigurierbar.
 Universelle GPU-Erkennung	NVIDIA (nvidia-smi), AMD (rocm-smi), Intel und Apple Silicon (WMI / lspci / sysctl). Mixed-Architecture-Systeme werden unterstützt.

Header-Steuerelemente

KV-Reihe	Sechs KV-Cache-Modi: f16, q8 ₀ +t4, t3/t3, t4/t4, q8 ₀ +t3, q8 ₀ /q8 ₀ . Klick zum Auswählen; VRAM-Balken und Fit-Anzeigen aktualisieren sich sofort.
Q / N / C (NEU v0.50)	Drei kleine Mode-Buttons zwischen der KV-Reihe und dem Probe-Button. Ausgegraut bis ein Probe-Profil für die aktuelle (Modell, Server)-Kombination existiert. Klick wendet das entsprechende (KV, LA)-Anker-Tupel an. Manuelle

	KV/LA-Klicks setzen den aktiven Mode zurück.
 Probe (NEU v0.50)	Öffnet den Probe Model-Dialog. Führt den Drei-Phasen-qnut-Workflow gegen das aktuell ausgewählte Modell auf dem aktiven Server-Slot durch. Read-only — das Modell wird nie verändert, nur beobachtet. Dauer: ~10–25 Minuten einmalig pro (Modell, Server). Ergebnisse werden als Profil gespeichert.
Port	TCP-Port für llama-server (Standard 8080). Vor dem Start hier ändern.
Ctx	Kontextfenster in Tokens. Leer = Modell-Standard verwenden. Jeder Wert wird als -c <ctx> an llama-server übergeben.
Timeout	Timeout pro Lauf für llama-bench in Sekunden (Standard 90). Verhindert Hänger in Bench-All-Matrizen.
☐ No Thinking	Reasoning- / Thinking-Modus deaktivieren via --reasoning off. Nützlich für Tool-Calling-Modelle.
☐ Benchmark	Einzelkonfigurations-Modus. Doppelklick oder Run führt f16-Baseline + die aktuell gewählte KV-Config aus und schreibt eine Delta-%-Tabelle.
☐ Bench All	Aktiviert die volle KV × LA-Matrix. Alle KV- und LA-Buttons werden blau; Klick zum Abwählen einzelner. Run startet, Stop bricht nach dem aktuellen Lauf ab.
LA: off / 1 / 5 / 7	Layer-Adaptive-Modi: off (uniform), 1 (Boundary-V), 5 (erste4 + letzte4 bei q8_0), 7 (aggressiv). Bei Bench All einzeln togglebar.
Run / Stop	Run startet den Benchmark mit der aktuellen Auswahl. Stop bricht nach Abschluss des laufenden Runs ab.
● Status-Pille	Idle / Laden / Laufend. Klick während Laufend öffnet http://localhost:{port} im Browser.

Probe (qnut) — NEU in v0.50

Probe ist ein Read-only Differential-Messwerkzeug. Es verändert nichts am Modell oder an der Hardware — es lässt das Modell unter jeder KV+LA-Kombination laufen, beobachtet die Antworten und schreibt ein Profil. Nach dem Profil-Build werden die drei Q / N / C-Mode-Buttons in der Top-Bar aktiv und du kannst die besten Settings mit einem Klick anwenden.

Die zugrundeliegende Methodologie — komprimierte Konfigurationen gegen eine f16-Baseline zu vergleichen statt gegen hardcoded erwartete Antworten — ist eine etablierte Technik in der Quantization-Forschung und nicht originär zu QLauncher. Originär ist hier die konkrete Umsetzung: der Drei-Phasen-Workflow, die (KV, LA)-Anker-Tupel, die Profil-Speicherung pro (Modell, Server) und die Q / N / C-Mode-Button-UI.

Warum differential?

Der naive Ansatz wäre: dem Modell eine Frage stellen, die erwartete Antwort hardcoden, und prüfen ob das Modell sie geliefert hat. Das bricht bei kleinen oder stark quantisierten Modellen, die schon auf reinem f16 Schwierigkeiten haben — jede Config sieht kaputt aus, weil das Modell selbst nicht zuverlässig antworten kann. Der differentielle Ansatz nutzt die f16-Baseline stattdessen als Referenz: jede komprimierte Config wird gutgeschrieben wenn sie dieselbe Antwort liefert wie f16, unabhängig davon ob diese Antwort „richtig“ war. Ein dummes Modell ist konsistent dumm über alle Configs hinweg, und der Test erkennt die Configs korrekt als KV-transparent.

Die drei Phasen

Phase 0 Baseline	f16 + LA=0 — sammelt Antworten für alle Probes. Keine Bewertung; wir etablieren
-------------------------	---

	nur die Referenz. ~90 Sekunden.
Phase 1 KV-Sweep	5 verbleibende KV-Configs bei LA=0. Jede Antwort wird gegen die Baseline verglichen. Score 0–3 pro Config. ~7–8 Minuten.
Phase 2 LA-Sweep	Für jede KV mit Score ≥ 2 in Phase 1 werden auch die anderen LA-Modi (1, 5, 7) getestet. Plus f16 mit allen LAs um die Lookahead-Transparenz auf der Baseline zu verifizieren. ~9–18 Minuten.

Wie die Anker abgeleitet werden

Quality (Q)	Per Definition immer (f16, LA=0). f16 ist die Referenz-Baseline; nichts kann treuer sein als die Baseline selbst.
Neutral (N)	Das Tupel mit der stärksten Compression bei Score 3 (perfekter Baseline-Match auf allen Probes). Der Alltagsfahrer — maximale Compression die sich noch nicht von f16 unterscheiden lässt.
Context (C)	Das Tupel mit der stärksten Compression bei Score ≥ 2 (eine Probe darf abweichen). Verwende dies wenn du jedes Megabyte KV-Cache für langen Context brauchst. Quality ist leicht entspannt aber das Modell produziert noch brauchbaren Output.
degraded-Flag	Wenn kein Tupel in der gesamten Matrix Score 3 erreicht, wird der Lauf als degraded markiert und Neutral fällt auf den besten verfügbaren Score zurück. Der Dialog warnt dich wenn das passiert — typischerweise bedeutet das, dass das Modell klein / stark quantisiert ist und auf jegliche KV-Compression empfindlich reagiert.



Profil-Speicherung

Profile werden pro (Modell, Server-Slot) unter `cfg["nut_profiles"]` gespeichert. Dasselbe Modell auf verschiedenen Server-Builds (mainline / TheTom / spiritbuun / Gemma4) bekommt unabhängige Profile — der Build zählt, weil die KV-Cache-Implementierung zwischen Forks unterschiedlich ist. Der Slot-Wechsel über die Footer-Buttons re-evaluert die Q / N / C-Buttons automatisch.



Empfohlener Workflow

1) Modell hinzufügen und auswählen. 2) GPU-Zeile wählen die du nutzen willst. 3)  Probe klicken und warten — typisch 10–25 Minuten pro Modell+Server-Kombination. 4) Nach dem Lauf leuchten Q / N / C auf. 5) Ab jetzt einfach Q für Quality-Arbeit, N für Alltag, C für Long-Context-Arbeit klicken. Die KV- und LA-Buttons folgen sichtbar damit du lernst was der jeweilige Modus bedeutet. 6) Probe nur dann erneut laufen lassen, wenn du den Server-Build aktualisierst oder ein neues Modell hinzufügst.

Footer-Steuerelemente

Rescan Models	Alle konfigurierten Modell-Verzeichnisse jetzt neu scannen. Berücksichtigt das Recursive-Flag.
Update Binaries	Den eingebauten Binary-Updater für den aktiven llama-server-Ordner ausführen.
Slot-Buttons	Ein-Klick-Wechsel zwischen Lesezeichen für llama-server.exe-Builds. Der aktive Slot ist hervorgehoben. Slot-Wechsel re-evaluert auch das Probe-Profil für die neue

	(Modell, Server)-Kombination.
 Save Results...	Wird grün, sobald ein Benchmark fertig ist. Klick hängt die Ergebnistabelle an TurboQuant_Benchmark.md neben dem Launcher an.
Paths	Öffnet den Configure-Paths-Dialog: Modell-Verzeichnisse, Recursive-Flag, aktive llama-server.exe und Quick-Switch-Slots.
About	Version, Lizenz, Build-Info und Links zu den Upstream-Projekten.

Maus & Tastatur

Doppelklick Zeile	Server mit ausgewähltem Modell auf dieser GPU starten (oder stoppen, wenn bereits aktiv).
Einfachklick	Modell- / GPU-Zeile auswählen, ohne zu starten.
↑ ↓ / ← →	Modelle navigieren (hoch / runter) und GPUs innerhalb eines Modells (links / rechts).
Enter	Server für aktuell ausgewählte Zeile umschalten.
Escape	Laufenden Server stoppen.
Klick Status-Pille	http://localhost:{port} im Browser öffnen (nur wenn Laufend).
Splitter ziehen	Modellliste vs. Log-Bereich vergrößern / verkleinern.

KV-Cache-Konfigurationen

f16	Baseline. Voller float16-KV-Cache, keine Komprimierung. Dient als Referenz für Benchmark-Deltas und Probe-Baseline.
q8o+t4	K bei q8_0, V bei turbo4. Beste Allzweck-Komprimierung. ≈3,5× kleinerer KV-Cache, nahezu kein Qualitätsverlust.
t3/t3	Sowohl K als auch V bei turbo3. Aggressivste Variante, die bei den meisten Modellen die Qualität noch erhält. ≈4,5× kleiner.
t4/t4	Sowohl K als auch V bei turbo4. Maximale Komprimierung. Manche Modelle zeigen messbaren Qualitätsverlust — mit Probe-Lauf verifizieren.
q8o+t3	K bei q8_0, V bei turbo3. Konservativer Mittelweg.
q8o/q8o	Sowohl K als auch V bei q8_0. Die klassische Nicht-TurboQuant-8-Bit-Baseline zum Vergleich.



Nicht KV manuell wählen — Probe entscheiden lassen

Der ganze Sinn von v0.50's Probe + Q / N / C-Workflow ist, dass du dir nicht merken musst welche KV-Config für welches Modell am besten ist. Probe einmal pro (Modell, Server) laufen lassen, dann einfach Q, N oder C klicken. Die KV- und LA-Buttons bewegen sich sichtbar zur empfohlenen Position, und mit der Zeit entwickelst du ein Gefühl dafür welche Modelle welche Settings bevorzugen.

Open-Source-Projekte & Danksagungen

★ spiritbuun / llama.cpp	TurboQuant-CUDA-Implementierung · dieser Build · github.com/spiritbuun/llama-cpp-turboquant-cuda
★ TheTom / turboquant_plus	Alternativer TurboQuant-Build mit fused mmvq-Kernel · github.com/TheTom/turboquant_plus
★ ggml-org / llama.cpp	Upstream-llama.cpp-Projekt · github.com/ggml-org/llama.cpp
💬 Community-Diskussion	Benchmark-Ergebnisse · seanrasch · AmesianX · primoco et al. · github.com/ggml-org/llama.cpp/discussions/20969
🔬 Probe-Methodologie	Differentielles f16-Baseline-Testing ist eine etablierte Technik in der Quantization-Forschung. Das TurboQuant-Paper (Zandieh et al., ICLR 2026) und die laufende Arbeit in Discussion #20969 haben die v0.50-Implementierung beeinflusst.

💡 TurboQuant-Paper

Zandieh et al., „TurboQuant: KV Cache Compression via Walsh–Hadamard Rotation“, ICLR 2026, Google Research. Dieses Tool ist ein unabhängiges Community-Projekt und nicht mit Google verbunden.